

UCS503 – Software Engineering Project

Academic Year 2026–2027

Sports Complex Management System (SCMS)

*QR-Based Sports Equipment Booking, Fair Allocation and Inventory
Management*

Submitted by:

1024240025 Jaspreet Singh

1024240026 Sanveer Singh

1024240028 Harshjot Singh

B.E. Third Year, Computer Science & Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Software Engineering Project Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Problem Statement	1
1.3	Project Objectives	1
1.4	Scope	2
2	Methodology and System Design	3
2.1	System Architecture	3
2.2	Technical Stack	3
2.3	Core Modules	3
3	Data Flow and Database Design	5
3.1	Context-Level Data Flow Diagram	5
3.2	DFD Level 1	5
3.3	Entity Relationship Design	6
3.4	Important Data Entities	6
3.4.1	Students	6
3.4.2	Equipment	7
3.4.3	Bookings	7
3.4.4	Transactions	7
3.4.5	Usage Statistics	7
3.4.6	Defaulters	7
4	Functional Design and Workflows	8
4.1	Use Case Diagram	8
4.2	Student Workflow	8
4.3	Staff Workflow	9
4.4	Admin Workflow	9
4.5	Booking and Allocation Workflow	9
5	Fairness, Business Rules and Security	10
5.1	Fairness-Based Allocation	10
5.2	Daily Usage Cap	10
5.3	Equipment State Management	10
5.4	No-Show and Overdue Rules	10
5.5	QR-Based Verification	11
5.6	Authentication and Authorisation	11
6	Implementation and Interface Design	12
6.1	Student Mobile Application	12
6.2	Staff Mobile Application	12
6.3	Admin Web Dashboard	12
7	Testing and Evaluation	14
7.1	Testing Strategy	14
7.2	Verification Results	14
7.3	Local End-to-End Environment	14
7.4	Testing of Critical Business Scenarios	15

8	Current Status, Deployment and Future Work	16
8.1	Current Implementation Status	16
8.2	Deployment Requirements	16
8.3	Future Work	16
9	Conclusion	18
9.1	Summary of Work	18
9.2	Key Contribution	18
9.3	Final Remarks	18

Chapter 1

Introduction

1.1 Background and Context

Managing sports equipment in a university sports complex involves more than simply recording who borrowed an item. Students need to know equipment and slot availability, submit booking requests, receive fair allocation, and complete collection and return procedures. Staff need reliable verification, issue/return records, inventory visibility, and damage tracking. Administrators need control over equipment, booking slots, fairness rules, staff accounts, defaulters, and analytics.

The **Sports Complex Management System (SCMS)** is designed as a full-stack system for digitising this workflow. Students and staff use a Flutter mobile application, while administrators use a React web dashboard. A FastAPI backend provides the central business logic and connects the applications to a PostgreSQL database.

1.2 Problem Statement

Traditional or manually managed sports equipment booking can lead to several operational problems:

- Lack of real-time visibility of equipment and slot availability.
- Manual handling of booking requests and allocation.
- Unequal access when many students request limited-capacity slots.
- Difficulty verifying students and equipment during issue and return.
- Incomplete tracking of overdue, damaged, or returned equipment.
- Limited visibility of usage statistics and operational performance.

SCMS addresses these problems through centralised booking, configurable fairness-based allocation, QR verification, transaction tracking, inventory management, notifications, and an administrative analytics dashboard.

1.3 Project Objectives

The major objectives of SCMS are:

1. Provide students with a digital platform to browse equipment and available slots.
2. Allow students to submit and track equipment booking requests.
3. Allocate limited-capacity slots using a transparent fairness-based priority mechanism.
4. Provide QR-based student and equipment verification for issue and return operations.
5. Maintain a central inventory with equipment status and condition.
6. Record every equipment issue and return as a transaction.
7. Identify overdue users and maintain defaulter records.

8. Provide in-app notifications for booking, waitlist, return, and overdue events.
9. Provide administrators with inventory, bookings, transactions, defaulters, staff, fairness, and analytics management.
10. Provide a deployable architecture that can be piloted with real-time sports-complex operations.

1.4 Scope

The system covers three main user-facing interfaces:

- **Student Mobile Application:** registration, login, equipment browsing, slot booking, booking status, QR identity, history, notifications, and profile.
- **Staff Mobile Application:** QR scanning, student validation, equipment issue, return processing, damage logging, queue viewing, and inventory status.
- **Admin Web Dashboard:** dashboard, analytics, inventory, slots, bookings, transactions, defaulters, fairness configuration, and staff management.

The backend is the single source of truth for authentication, authorisation, booking rules, fairness allocation, transactions, inventory state, notifications, and scheduled operations.

Chapter 2

Methodology and System Design

2.1 System Architecture

SCMS follows a client-server architecture with separate interfaces for students, staff, and administrators. Both mobile and web clients communicate with a central FastAPI backend. The backend communicates with PostgreSQL for persistent data storage.

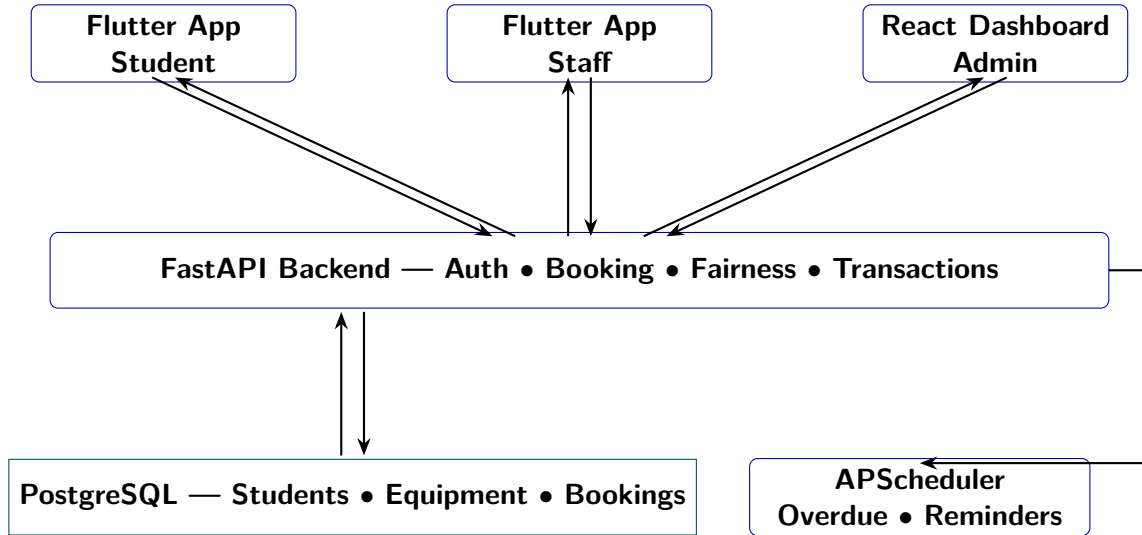


Figure 2.1: High-Level SCMS System Architecture

2.2 Technical Stack

The project uses the following technologies:

Layer	Technology / Purpose
Mobile	Flutter 3, Dart, Riverpod, GoRouter, Dio
Web	React 18, TypeScript 5, Vite 5, TanStack Query, Axios
Backend	Python, FastAPI 0.115
Database	PostgreSQL, SQLAlchemy 0.0.21
Migrations	Alembic 1.13
Authentication	JWT (HS256), secure password hashing
QR	qr_flutter for QR display, mobile_scanner for scanning
Storage	flutter_secure_storage for mobile authentication data
Scheduling	APScheduler 3.10
Charts	Recharts 2
Web UI	Tailwind CSS 3, Radix UI, Lucide
Testing	pytest, Vitest, flutter_test
Deployment	Docker, nginx

Table 2.1: SCMS Technology Stack

2.3 Core Modules

The backend is organised around the following modules:

1. **Authentication:** Student registration/login, staff login, JWT validation, role-based access, and session restoration.
2. **Students:** Student records, QR identity, status, and usage information.
3. **Equipment and Inventory:** Equipment records, categories, quantities, status, condition, and administrative CRUD operations.
4. **Slots:** Admin-created booking slots with capacity, timing, cutoff, and status.
5. **Bookings:** Booking requests, confirmation, waitlisting, cancellation, queue position, and priority information.
6. **Fairness Engine:** Batch allocation using recent usage and recency-based priority.
7. **Transactions:** Equipment issue, return, damage recording, and transaction history.
8. **Notifications:** In-app booking, waitlist, return, overdue, and damage notifications.
9. **Analytics:** Booking trends, utilisation, equipment usage, damage statistics, and operational KPIs.
10. **Scheduler:** Automated slot cutoffs, no-show detection, overdue detection, and return reminders.

Chapter 3

Data Flow and Database Design

3.1 Context-Level Data Flow Diagram

At the highest level, SCMS is treated as a single process interacting with external entities. The main external entities are Student, Staff, Admin, and the Notification Service.



Figure 3.1: Context-Level Data Flow Diagram

3.2 DFD Level 1

The Level 1 DFD decomposes SCMS into its major internal processes:

- 1.0 Registration & Login
- 2.0 Booking & Scheduling
- 3.0 Fairness Compute
- 4.0 Issue/Return Transactions
- 5.0 Inventory Management
- 6.0 Notifications & Analytics

The principal data stores are Students (D1), Equipment Items (D2), Bookings (D3), Transactions (D4), Usage Statistics (D5), and Defaulters (D6).

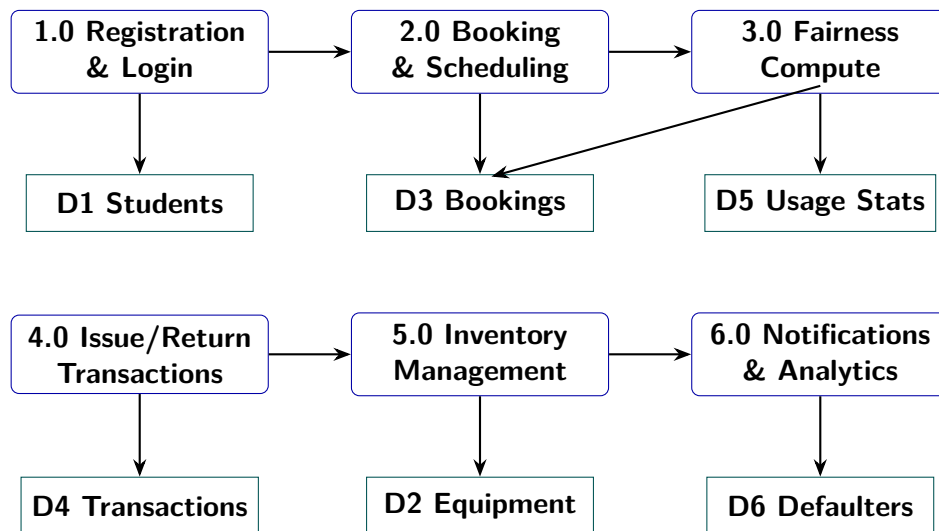


Figure 3.2: DFD Level 1: Major SCMS Processes and Data Stores

3.3 Entity Relationship Design

The database is centred around students, equipment, bookings, transactions, usage statistics, and defaulters. The core relationships are:

- A student **creates** bookings.
- A student **makes** transactions.
- A booking **reserves** an equipment item.
- A transaction records an equipment item being issued.
- A student **has** usage statistics.
- A student **may become** a defaulter.

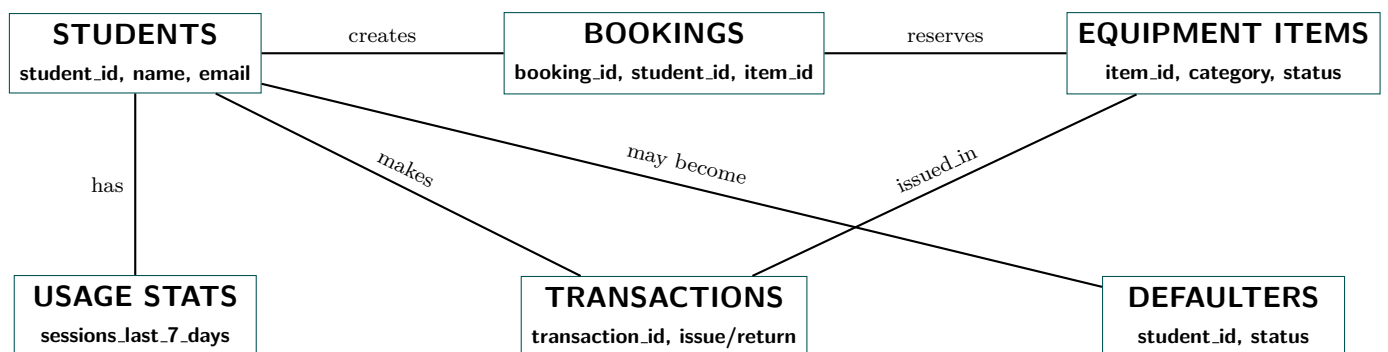


Figure 3.3: Simplified Entity Relationship Design

3.4 Important Data Entities

3.4.1 Students

Stores student identity and account information, including student ID, name, email, department, QR identity information, and account status.

3.4.2 Equipment

Stores equipment identity, category, quantity, status, and condition. Equipment states include AVAILABLE, BOOKED, ISSUED, MAINTENANCE, and RETIRED.

3.4.3 Bookings

Stores student booking requests, selected equipment/slot information, booking status, timing, and the priority score associated with allocation.

3.4.4 Transactions

Stores physical issue and return activity, including student, equipment, issue time, due time, return time, and return condition.

3.4.5 Usage Statistics

Stores recent student usage such as sessions and total minutes over the last seven days. These values support fairness calculation.

3.4.6 Defaulters

Stores active or resolved default records, including late-return information and current defaulter status.

Chapter 4

Functional Design and Workflows

4.1 Use Case Diagram

The Use Case model identifies four actors: Student, Staff, Admin, and the System Notification Service.

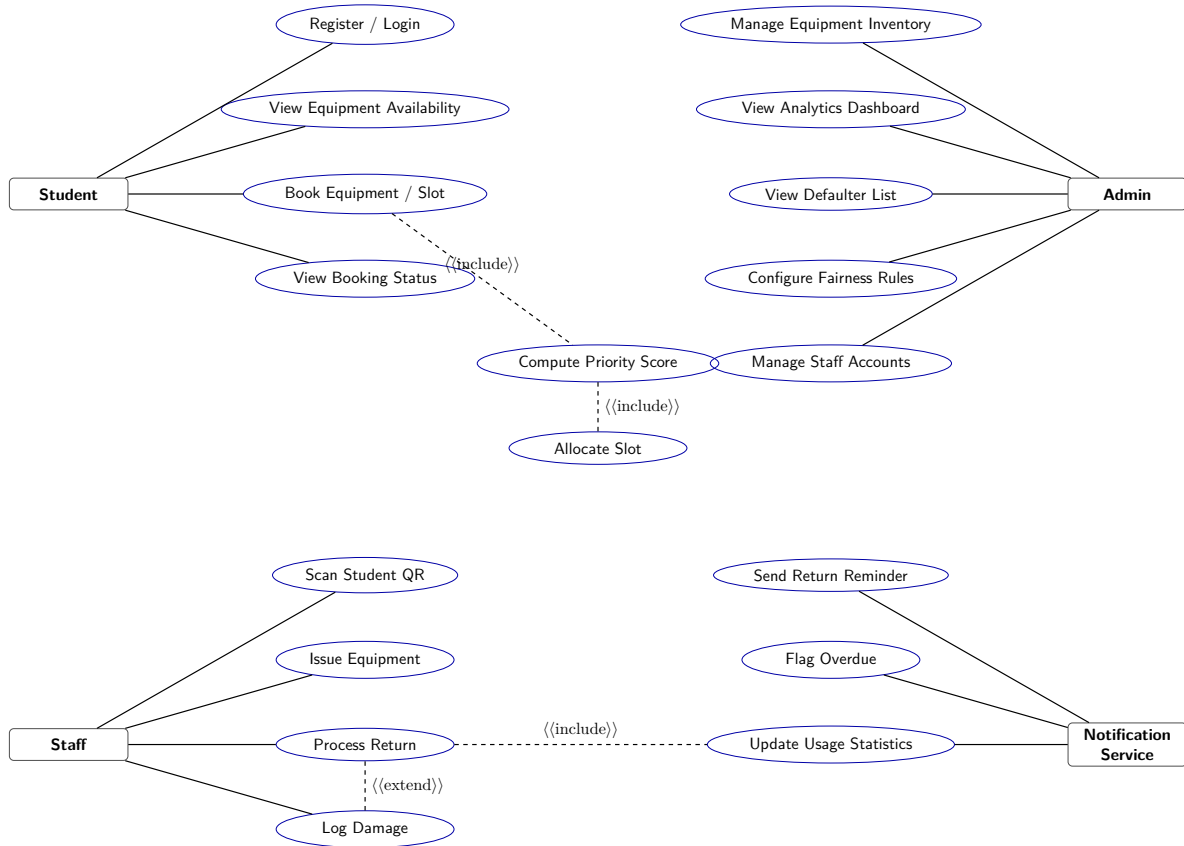


Figure 4.1: SCMS Use Case Diagram

4.2 Student Workflow

The student workflow is:

1. Register or log in.
2. Browse available sports equipment.
3. Select an available booking slot.
4. Submit a booking request.
5. Wait for the slot cutoff and fairness-based allocation.
6. Receive a **CONFIRMED** or **WAITLISTED** result.
7. View the booking and QR identity information.
8. Collect the equipment through staff verification and QR scanning.

9. Return the equipment through the staff-assisted return process.
10. View transaction history and notifications.

4.3 Staff Workflow

Staff performs the operational equipment-counter workflow:

1. Open the staff queue or scan interface.
2. Scan the student's QR code.
3. Verify the student and confirmed booking.
4. Scan the equipment QR code.
5. Issue the equipment and create the transaction.
6. When the equipment is returned, process the return.
7. Record the equipment condition.
8. If damaged, log the damage and place the equipment into maintenance.

4.4 Admin Workflow

The administrator manages the overall system:

1. Manage equipment inventory.
2. Create and manage booking slots.
3. Monitor bookings and transactions.
4. Review defaulters.
5. Configure fairness parameters.
6. Manage staff/admin accounts.
7. Monitor analytics and utilisation.

4.5 Booking and Allocation Workflow

The booking lifecycle is:

REQUESTED → CONFIRMED / WAITLISTED → COMPLETED / CANCELLED / NO_SHOW

Allocation is performed in batch after the booking cutoff. Confirmed bookings occupy the available slot capacity; remaining eligible requests are waitlisted.

If a confirmed booking is cancelled or becomes a no-show, the next eligible waitlisted booking can be promoted.

Chapter 5

Fairness, Business Rules and Security

5.1 Fairness-Based Allocation

SCMS uses a configurable fairness mechanism to avoid repeatedly favouring students who have already received substantial recent usage.

The priority score is:

$$\text{Priority Score} = S_{norm} + P_{recency}$$

where:

$$S_{norm} = \frac{\text{sessions in last 7 days}}{\max(\text{sessions in last 7 days across students})}$$

The recency component is:

$$P_{recency} = \begin{cases} (2 - d) \times 0.5, & d < 2 \\ 0, & d \geq 2 \end{cases}$$

where d represents the number of days since the student's last session.

A **lower score receives higher allocation priority**. This means the algorithm considers both recent usage and recency while maintaining a configurable daily usage cap.

5.2 Daily Usage Cap

The default daily cap is:

$$2 \text{ sessions per student per day}$$

The value is configurable by an administrator. Students who reach the applicable cap are not allocated additional sessions under the fairness rules.

5.3 Equipment State Management

Equipment follows controlled states:

$$\text{AVAILABLE} \rightarrow \text{BOOKED} \rightarrow \text{ISSUED} \rightarrow \text{AVAILABLE}$$

If returned damaged:

$$\text{ISSUED} \rightarrow \text{RETURNED_DAMAGED} \rightarrow \text{MAINTENANCE}$$

This prevents damaged equipment from being immediately issued again.

5.4 No-Show and Overdue Rules

A confirmed booking can be marked as NO_SHOW if no transaction is recorded within the defined no-show window after the slot begins.

The scheduler also identifies overdue transactions and creates the relevant reminder or overdue records.

An active defaulter is prevented from creating a new booking according to the business rules. Defaulter status is separate from the administrative student status; becoming a defaulter does not automatically mean the student account is suspended.

5.5 QR-Based Verification

SCMS uses QR codes for operational verification.

The staff workflow verifies:

1. Student identity.
2. Booking eligibility.
3. Equipment identity.
4. Issue/return transaction.

This reduces manual entry and links the physical equipment operation directly to the digital transaction record.

5.6 Authentication and Authorisation

Authentication uses JWT-based sessions with secure password hashing. The backend enforces role-based access so that:

- Students can access student resources.
- Staff can access operational issue/return resources.
- Admins can access administrative management resources.

The backend remains the authority for permissions; the mobile or web interface does not independently decide whether an operation is allowed.

Chapter 6

Implementation and Interface Design

6.1 Student Mobile Application

The student application is built using Flutter with Riverpod for state management and GoRouter for navigation.

The major student screens are:

Screen	Purpose
Splash	Restores the authentication session and routes the user.
Login	Student/staff credential entry.
Register	New student registration.
Home	Upcoming bookings and notification status.
Equipment	Browse available equipment.
Equipment Detail	View equipment details, choose slot, and book.
My QR	Display the student's QR identity.
Bookings	View active bookings and statuses.
Booking Detail	View booking details, queue position, and cancellation.
History	View completed transactions.
Transaction Detail	View issue/return details and condition.
Notifications	View read/unread in-app notifications.
Profile	View account information and usage statistics.

Table 6.1: Student Mobile Application Screens

6.2 Staff Mobile Application

The staff interface provides the operational workflow:

Screen	Purpose
Staff Home	Today's queue and open transactions.
Scan	QR camera for operational scanning.
Student Validation	Verify student and booking after scanning.
Issue Confirmation	Confirm equipment issue.
Return Process	Process return and record condition/damage.
Queue	View confirmed bookings and open transactions.
Inventory	View equipment status.
Profile	Staff account and logout.

Table 6.2: Staff Mobile Application Screens

6.3 Admin Web Dashboard

The web dashboard is built using React, TypeScript, Vite, TanStack Query, Axios, Tailwind CSS, Radix UI, Recharts, and Lucide icons.

Screen	Purpose
Login	Staff/admin authentication.
Dashboard	Operational KPI summary and quick actions.
Analytics	Trends, utilisation, peak hours, and damage statistics.
Inventory	Equipment CRUD and status management.
Slots	Create/manage booking slots, capacity, and cutoffs.
Bookings	Review, filter, inspect, and cancel bookings.
Transactions	Review issue and return transactions.
Defaulters	Review and resolve defaulter records.
Fairness	Configure daily cap and fairness parameters.
Staff	Create, edit, and deactivate staff/admin accounts.

Table 6.3: Admin Web Dashboard Screens

Chapter 7

Testing and Evaluation

7.1 Testing Strategy

Testing was performed separately for the backend, web dashboard, and Flutter application.

1. **Backend Unit and Integration Testing:** API, authentication, booking, fairness, transaction, inventory, notification, and scheduler behaviour.
2. **Web Testing:** Component and page-level tests using Vitest and Testing Library.
3. **Flutter Testing:** Widget/application tests together with static analysis.
4. **Build Verification:** Production web build and Android debug APK build verification.
5. **Runtime Verification:** Local PostgreSQL database, Alembic migrations, seeded demo data, backend API, and web dashboard integration.

7.2 Verification Results

The current implementation has been verified using the following project-level results:

Component	Result	Verification
Backend	26/26 tests passed	pytest suite green
Web Dashboard	69/69 tests passed	Vitest suite green
Web TypeScript	0 errors	Type checking successful
Web Production Build	Successful	Vite build completed
Flutter	28/28 tests passed	flutter test suite green
Flutter Static Analysis	0 issues	flutter analyze successful
Database Migration	Successful	Alembic initial schema applied
Demo Data	Successful	Seed data inserted and verified

Table 7.1: SCMS Verification Summary

7.3 Local End-to-End Environment

The local deployment consists of:

- PostgreSQL database for persistent storage.
- FastAPI backend running on port 8000.
- React web dashboard running through Vite during development.
- Flutter application connected to the backend using the appropriate local/emulator base URL.
- Alembic for database schema migration.
- Seed data for demonstrating the booking and fairness workflow.

A representative fairness demonstration uses a capacity-one slot with students having different recent usage. The fairness engine assigns the lower priority score to the student with greater allocation priority and places the other eligible request on the waitlist.

7.4 Testing of Critical Business Scenarios

The following scenarios are important for acceptance testing:

1. Student registration and login.
2. Student equipment and slot browsing.
3. Booking request creation.
4. Fairness-based confirmation and waitlisting.
5. Booking cancellation and waitlist promotion.
6. Student QR verification by staff.
7. Equipment QR verification and issue.
8. Normal return and equipment availability update.
9. Damaged return and maintenance status update.
10. Overdue detection and defaulter creation.
11. In-app notification generation.
12. Admin inventory, booking, transaction, fairness, and analytics operations.

Chapter 8

Current Status, Deployment and Future Work

8.1 Current Implementation Status

The project currently has the following implementation status:

Area	Status
Backend	Complete; routers, services, scheduler, migrations, and tests implemented.
Web Dashboard	Complete; administrative screens and analytics implemented.
Student Flutter App	Complete; student workflow and screens implemented.
Staff Flutter App	Complete; staff operational workflow implemented.
Automated Testing	All current project suites green.
Docker Deployment	Dockerfile and docker-compose prepared for deployment.
Real-Time Sports Complex Pilot	Next major step: deployment with institutional infrastructure and real operational data.

Table 8.1: Current SCMS Implementation Status

8.2 Deployment Requirements

Before production deployment, the following configuration steps are required:

1. Configure a strong production `SECRET_KEY`.
2. Configure the production PostgreSQL database.
3. Run the Alembic migrations using `alembic upgrade head`.
4. Configure production CORS origins.
5. Set the backend environment to production.
6. Configure the Flutter application to use the production HTTPS backend URL.
7. Deploy the web dashboard behind nginx or the selected production reverse proxy.
8. Create controlled administrator and staff accounts.

8.3 Future Work

The immediate next phase is a controlled pilot deployment at the Thapar Sports Complex. The pilot can validate the system using real equipment, slots, students, staff operations, and transaction data.

Future enhancements may include:

1. Production cloud deployment and monitoring.

2. Push notifications in addition to the current in-app notification mechanism.
3. More detailed operational and usage analytics.
4. Additional sports and resource categories.
5. Integration with institutional authentication or student systems where permitted.
6. Improved reporting and export facilities.
7. Operational feedback-driven refinements after the initial sports-complex pilot.

Chapter 9

Conclusion

9.1 Summary of Work

The **Sports Complex Management System (SCMS)** provides an integrated digital workflow for sports equipment management. The system connects students, staff, administrators, and notification functionality through a common FastAPI backend and PostgreSQL database.

The project combines:

- Digital equipment and slot booking.
- Fairness-based batch allocation.
- QR-based identity and equipment verification.
- Issue and return transaction management.
- Inventory and damage management.
- Overdue and defaulter handling.
- In-app notifications.
- Administrative analytics and management.

9.2 Key Contribution

The main contribution of SCMS is the integration of the **booking, fairness, physical issue/return, inventory, and administrative management workflows** into a single system. Instead of treating booking and equipment handling as separate processes, the system connects the complete lifecycle from a student's request to final equipment return.

9.3 Final Remarks

SCMS has reached a functional and tested state suitable for a controlled real-time pilot. The next stage is to work with the Thapar Sports Complex administration to configure institutional infrastructure, validate operational requirements, and deploy the system in a controlled environment.

Bibliography

- [1] FastAPI Documentation, “FastAPI: Modern, Fast Web Framework for Building APIs with Python,” 2026.
- [2] Flutter Documentation, “Flutter: Build apps for any screen,” 2026.
- [3] React Documentation, “React: The library for web and native user interfaces,” 2026.
- [4] PostgreSQL Documentation, “PostgreSQL: The World’s Most Advanced Open Source Relational Database,” 2026.
- [5] Alembic Documentation, “Database Migration Tool for SQLAlchemy,” 2026.
- [6] SQLAlchemy Documentation, “SQL databases in Python, designed for simplicity, compatibility, and robustness,” 2026.
- [7] J. Singh, S. Singh, and H. Singh, “Sports Complex Management System (SCMS),” Thapar Institute of Engineering and Technology, Software Engineering Project, 2026.